[ ]:

Practical No. 2

Experiment: Diagrammatic representation and interpretation of statistical data.

Question 1

The following table shows the number of students enrolled in different departments of a college.

Frequency Table

Department	Number of Students
AI & DS	60
Computer Science	45
IT	50
Cyber Security	35
Electronics	40

Write a Python program to:

1. Store the given data in Python lists.
2. Draw a Simple Bar Chart using matplotlib.
3. Give a suitable title to the graph.
4. Label the X-axis and Y-axis, add legend, add grid.
5. Display the graph

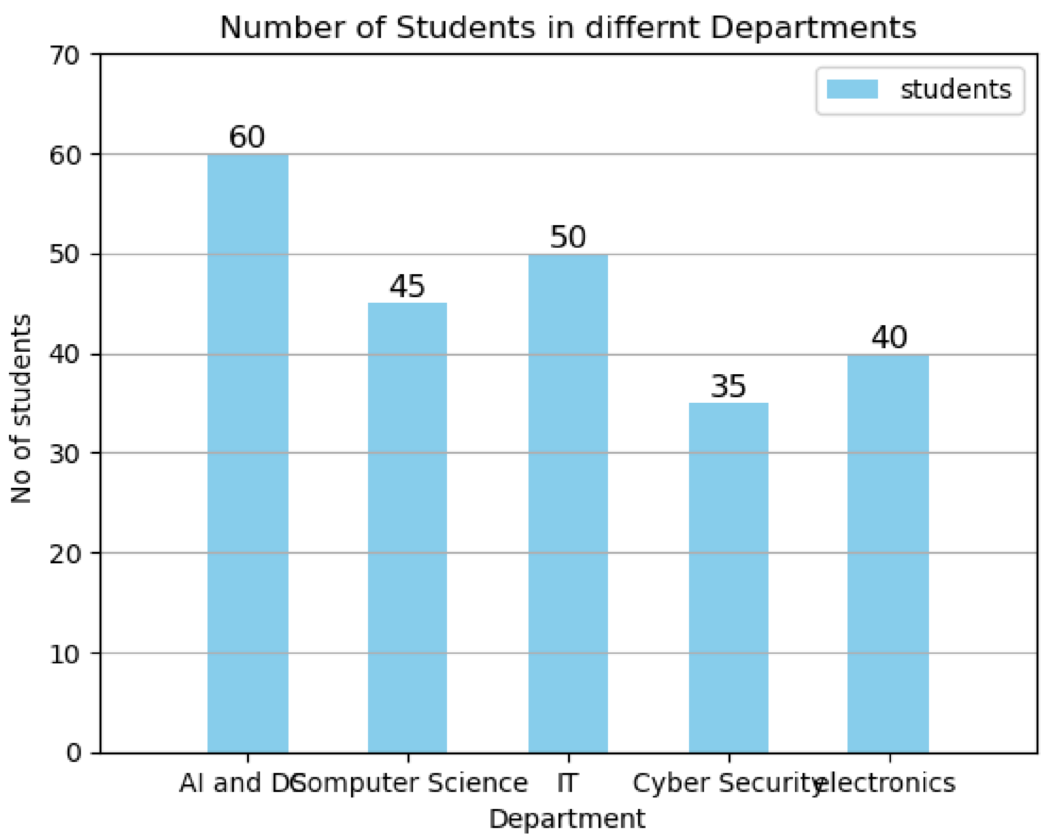
[3]:

```
import matplotlib.pyplot as plt

department= [ "AI and DS " ,"Computer Science","IT", "Cyber Security","electronics " ]
students = [ 60,45, 50, 35,40 ]
 bar_1= plt.bar(department,students,width= 0.5 , color = "skyblue", label="students")

plt.title (" Number of Students in diffrent Departments ")
plt.xlabel ( "Department")
plt.ylabel ("No of students ")
plt.legend()
plt.grid( axis="y")
plt.style.use('classic')
plt.ylim(0,70)
plt.margins(x=0.15)
plt.bar_label(bar_1)
plt.show()
```

```
Matplotlib is building the font cache; this may take a moment.
```



[ ]:

Question 2:

A Data Science institute recorded the number of students who enrolled in Online and Offline training programs during the year 2025.

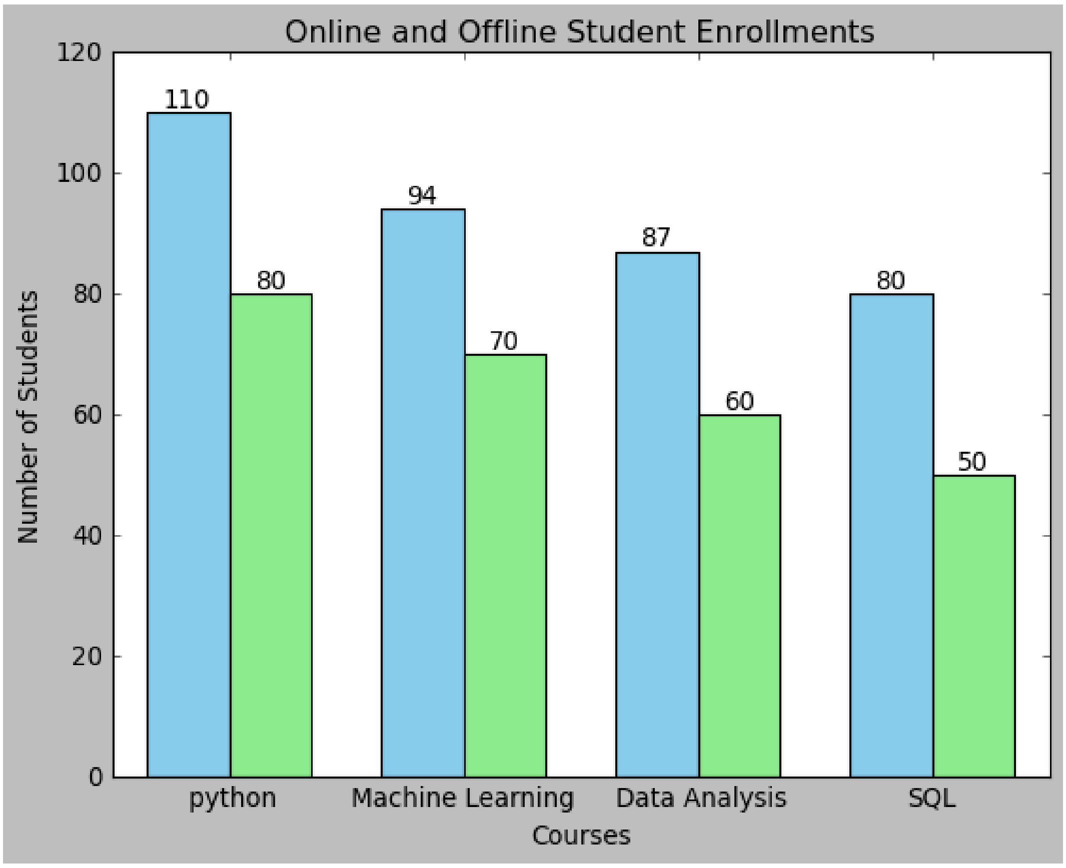
Table

Course	Online Students	Offline Students
Python	120	80
Machine Learning	90	70
Data Analytics	100	60
SQL	80	50

Write a Python program to:

1. Store the given data in Python lists.
2. Draw a Multiple Bar Chart to compare Online and Offline student enrollments.
3. Give a suitable title to the chart.
4. Label the X-axis and Y-axis.
5. Display the legend.
6. Display the graph.

[6]:
```
import matplotlib . pyplot as plt
import numpy as np
course = [ " python", " Machine Learning ", " Data Analysis", "SQL"]
online = [ 110,94,87,80]
offline=[80,70,60,50]
x=np.arange( len( course))
width=0.35
bar1=plt.bar (x- width/2,online,width,label="Online",color='skyblue')
bar2=plt.bar(x + width/2 , offline,width,label= "offline",color= "lightgreen" )
plt.bar_label(bar1)
plt.bar_label(bar2)
plt.xticks(x,course)
plt.title("Online and Offline Student Enrollments")
plt.xlabel("Courses")
plt.ylabel("Number of Students")
plt.show()
```



[ ]:

Question 3: Pie Chart

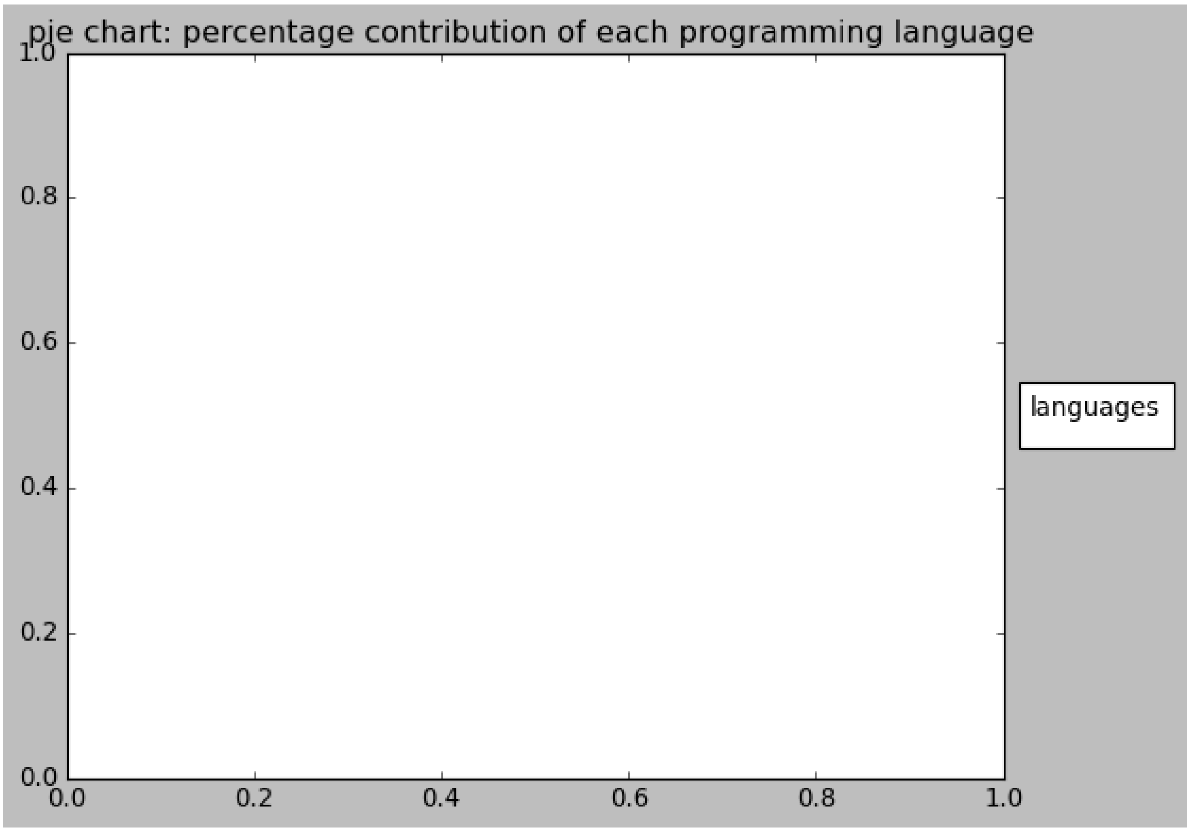
Represent the following data using a Pie Chart. Give an appropriate title and display the percentage contribution of each programming language on the chart.

Programming Language	Number of Students
Python	40
R	20

Programming Language	Number of Students
Java	15
C++	10
JavaScript	15

[12]:
```
import matplotlib.pyplot as plt

language = [ 'python','R','Java','C++','JavaScript' ]
students = [40,20,15,10,15]
plt.pie(students,
plt.title("pie chart: percentage contribution of each programming language ")
plt.legend( language , title= 'languages', loc = 'center left' , bbox_to_anchor=(1,0.5))
plt.show()
```



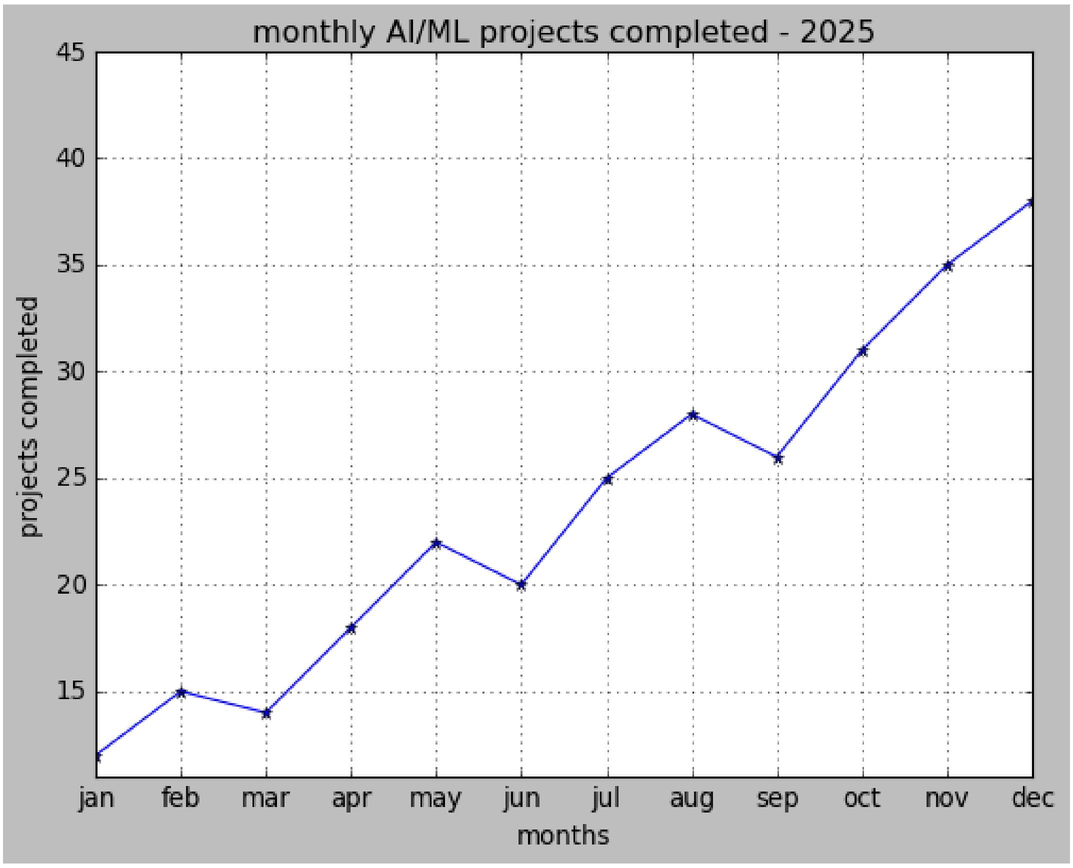
[ ]:

Question 4: Line Chart

A Data Science team recorded the number of AI/ML projects completed each month during 2025. The data are given below:

Month	Projects Completed
Jan	12
Feb	15
Mar	14
Apr	18
May	22
Jun	20
Jul	25
Aug	28
Sep	26
Oct	31
Nov	35
Dec	38

[18]:
```
import matplotlib.pyplot as plt
months=['jan','feb','mar','apr', 'may','jun','jul','aug','sep','oct','nov','dec']
projects=[12,15,14,18,22,20,25,28,26,31,35,38]
plt.plot(months,projects,marker='*')
for i in range (len(months)):
 plt.title('monthly AI/ML projects completed - 2025')
 plt.xlabel('months')
 plt.ylabel('projects completed')
 plt.ylim(min(projects)-1,45)
 plt.grid()
 plt.show()
```


monthly AI/ML projects completed - 2025
projects completed
months
45
40
35
30
25
20
15
jan
feb
mar
apr
may
jun
jul
aug
sep
oct
nov
dec


monthly AI/ML projects completed - 2025
projects completed
months
45
40
35
30
25
20
15
0.0
0.2
0.4
0.6
0.8
1.0


monthly AI/ML projects completed - 2025
projects completed
months
45
40
35
30
25
20
15
0.0
0.2
0.4
0.6
0.8
1.0


monthly AI/ML projects completed - 2025
projects completed
months
45
40
35
30
25
20
15
0.0
0.2
0.4
0.6
0.8
1.0


monthly AI/ML projects completed - 2025
projects completed
45
40
35
30
25
20
15
0.0
0.2
0.4
0.6
0.8
1.0
months


monthly AI/ML projects completed - 2025
projects completed
45
40
35
30
25
20
15
0.0
0.2
0.4
0.6
0.8
1.0
months


monthly AI/ML projects completed - 2025
projects completed
months
45
40
35
30
25
20
15
0.0
0.2
0.4
0.6
0.8
1.0


monthly AI/ML projects completed - 2025
projects completed
months
45
40
35
30
25
20
15
0.0
0.2
0.4
0.6
0.8
1.0


monthly AI/ML projects completed - 2025
projects completed
months
45
40
35
30
25
20
15
0.0
0.2
0.4
0.6
0.8
1.0


monthly AI/ML projects completed - 2025
projects completed
months
45
40
35
30
25
20
15
0.0
0.2
0.4
0.6
0.8
1.0

[ ]:

[ ]:

Practical 3: Graphical representation and interpretation of statistical data

Histogram

Frequency Polygon

Frequency Curve

Less than CF Curve

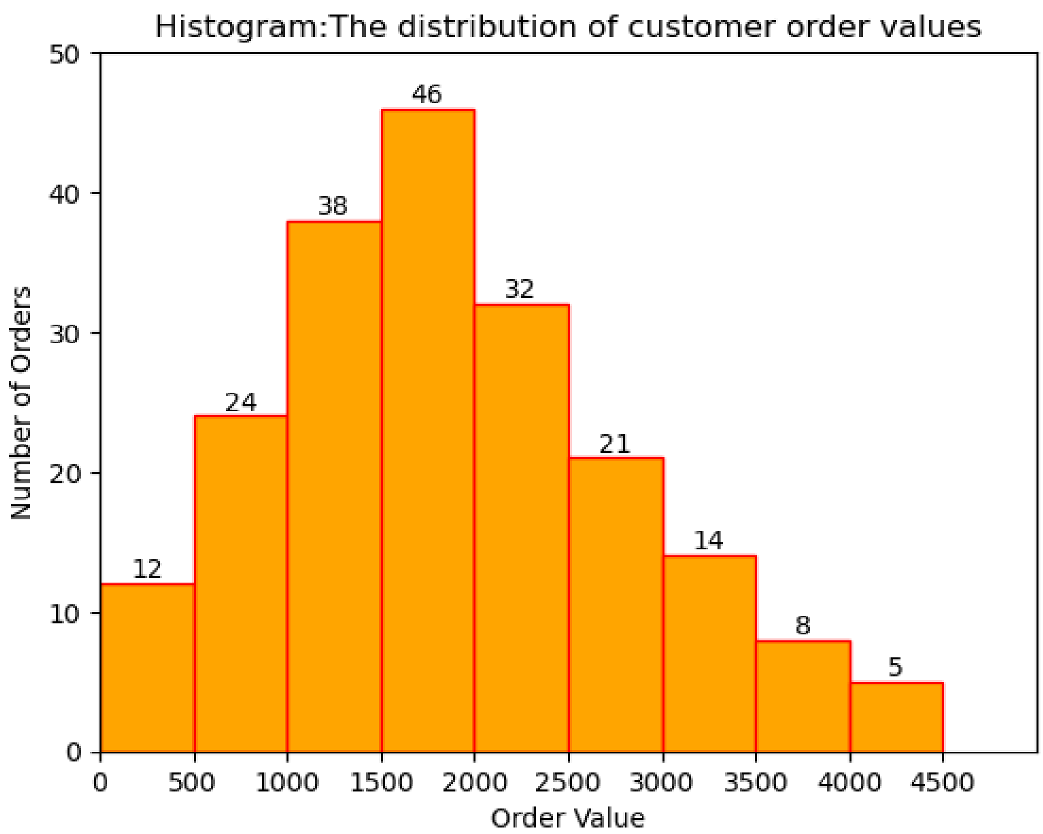
Question1 : Histogram — E-commerce Order Value

An e-commerce company analyzed the order values (₹) of 200 customer purchases made during a recent online sales campaign. The data were grouped as follows:

Order Value (₹)	Number of Orders
0–500	12
500–1000	24
1000–1500	38
1500–2000	46
2000–2500	32
2500–3000	21
3000–3500	14
3500–4000	8
4000–4500	5

Construct a Histogram to represent the distribution of customer order values. Give an appropriate title, label both axes, and display the class boundaries clearly on the X-axis.

[11]:
```
import matplotlib.pyplot as plt
lower = [i for i in range(0,4500,500)]
orders = [12,24,38,46,32,21,14,8,5]
hist = plt.bar(lower, orders, width = 500, align ='edge',
               color ='orange', edgecolor ='red')
plt.title('Histogram:The distribution of customer order values')
plt.xlabel('Order Value')
plt.ylabel('Number of Orders')
plt.xticks(range(0,5000,500))
plt.xlim(0,5000)
plt.ylim(0,50)
plt.bar_label(hist)
plt.show()
```



[ ]:

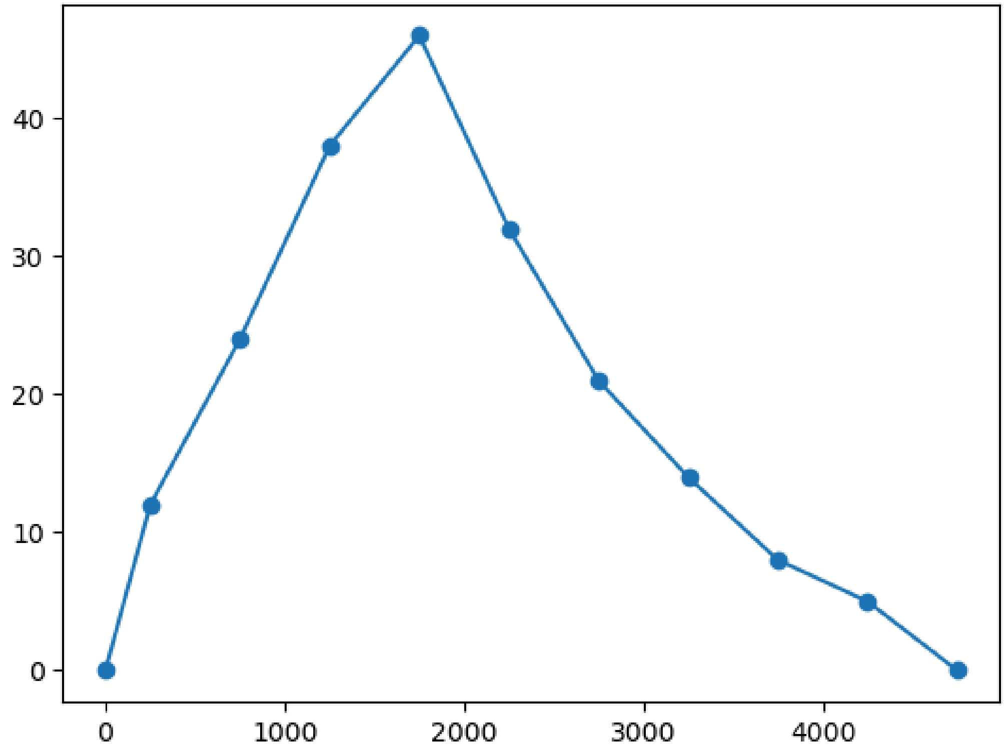
Question 2: Frequency Polygon:

Use same data above

[18]:
```
import matplotlib.pyplot as plt
lower=[i for i in range(0,4500,500)]
lower.append(4500)
midpoints = [int((lower[i]+lower[i+1])/2) for i in range(len(lower)-1)]
orders = [12,24,38,46,32,21,14,8,5]
# add one extra midpoints at both ends with 0 frequency:
```

```
midpoints = [0]+midpoints +[4750]
orders = [0] + orders+ [0]
plt.plot(midpoints, orders, marker = 'o')
for i in range(len(midpoints)):
    plt.text(midpoints[i],ordes[i]+1, str(orders[i]), ha = 'center')
plt.title('Frequency Polygon: The distribution of customer order values')
plt.xlabel('Midpoint : Order Values')
plt.ylabel('Number of Orders')
plt.xlim(0,5000)
plt.ylim(0,50)
plt.xticks(midpoints)
plt.show()
```

```
---------------------------------------------------------------------------NameError                                 Traceback (most recent ca
t)Cell In[18], line 11
      9 plt.plot(midpoints, orders, marker = 'o')
     10 for i in range(len(midpoints)):
---> 11     plt.text(midpoints[i],ordes[i]+1, str(orders[i]), ha = 'center')
     12 plt.title('Frequency Polygon: The distribution of customer order values')
     13 plt.xlabel('Midpoint : Order Values')
NameError: name 'ordes' is not defined
```

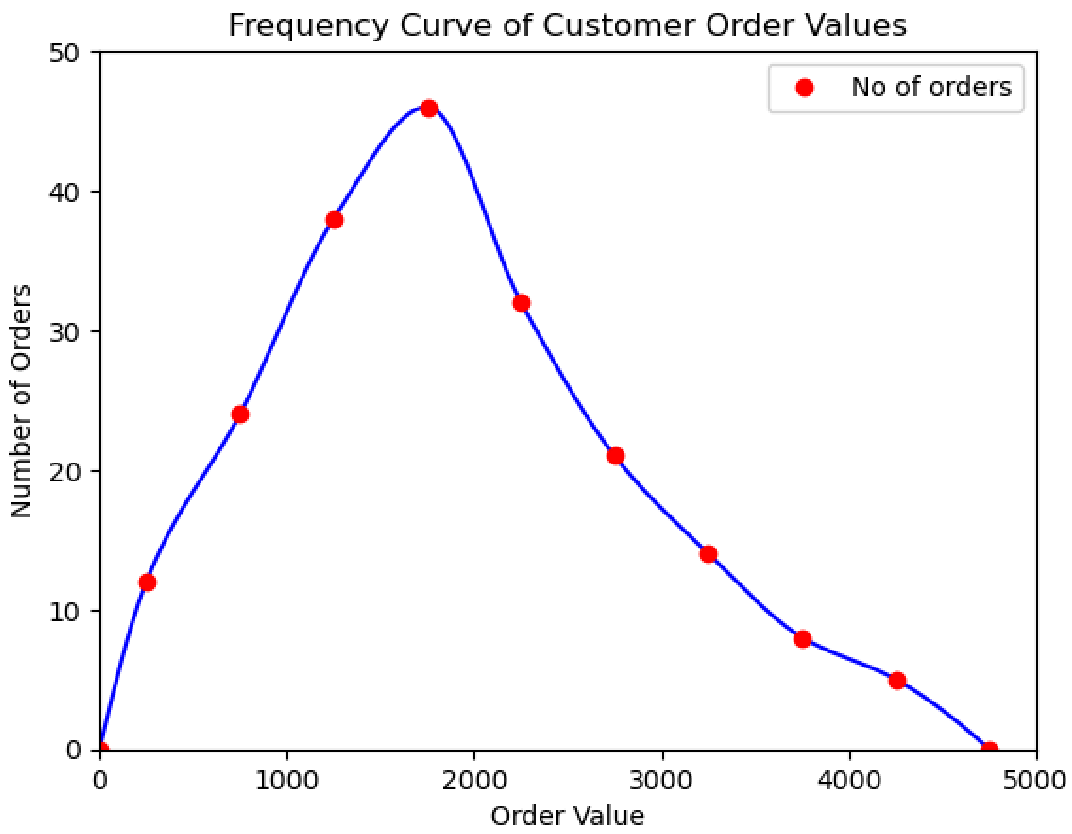


[ ]:

Question 3: Frequency Curve:

Use same data above

[24]:
```
import matplotlib.pyplot as plt
import numpy as np
from scipy.interpolate import PchipInterpolator
midpoints = [ 250, 750, 1250, 1750, 2250, 2750, 3250, 3750, 4250]
orders = [ 12,24, 38, 46, 32, 21, 14, 8, 5]
midpoints = [0]+ midpoints +[4750]
orders  = [0]+ orders + [0]
# Generate more x values for smooth curve
pchip = PchipInterpolator(midpoints, orders)
x_smooth = np.linspace (min(midpoints), max(midpoints), 300)
# Creat smooth curve
y_smooth = pchip(x_smooth)
plt.plot(x_smooth, y_smooth, color = 'blue')
plt.plot(midpoints, orders, 'o', color='red', linestyle = 'none', label = 'No of orders')
plt.xlabel('Order Value')
plt.ylabel('Number of Orders')
plt.title('Frequency Curve of Customer Order Values')
plt.xlim(0,5000)
plt.ylim(0,50)
plt.legend()
plt.show()
```



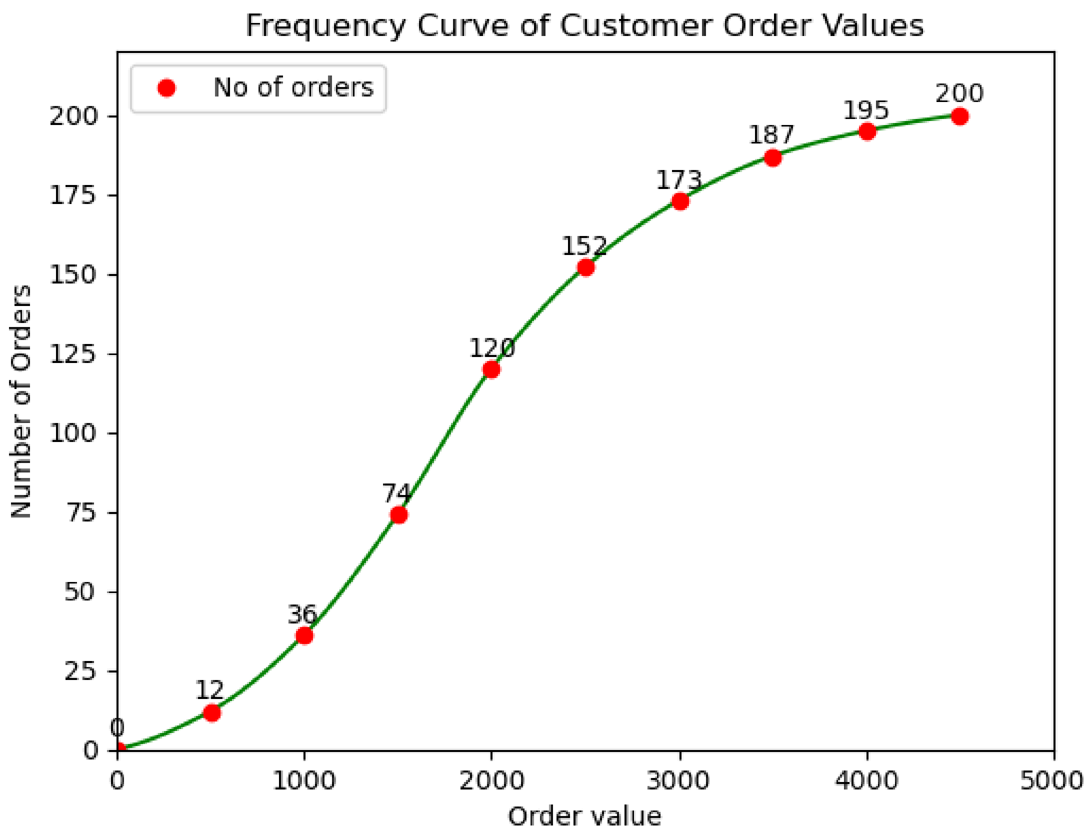
[ ]:

Question 4: Less than Cumulative Frequency Curve:

Use same data above

[10]:
```
import matplotlib.pyplot as plt
import numpy as np
from scipy.interpolate import PchipInterpolator
upper = [ i for i in range (500,5000,500)]
upper = [0] + upper
orders = [12,24,38,46,32,21,14,8,5]
orders =[0]+orders

cum_freq = np.cumsum(orders)
#generate more x values for smooth curve
pchip = PchipInterpolator( upper,cum_freq)
x_smooth = np.linspace(min(upper), max(upper), 500)
#create smooth curve
y_smooth = pchip(x_smooth)
plt.plot(x_smooth,y_smooth,color='green')
plt.plot(upper, cum_freq, 'o',color='red',linestyle ='none',
         label ='No of orders')
for i in range(len(upper)):
    plt.text(upper[i], cum_freq[i]+4, str(cum_freq[i]), ha='center')
plt.xlabel('Order value')
plt.ylabel('Number of Orders')
plt.title('Frequency Curve of Customer Order Values')
plt.xlim(0,5000)
plt.ylim(0,220)
plt.legend()
plt.show()
```



[ ]:

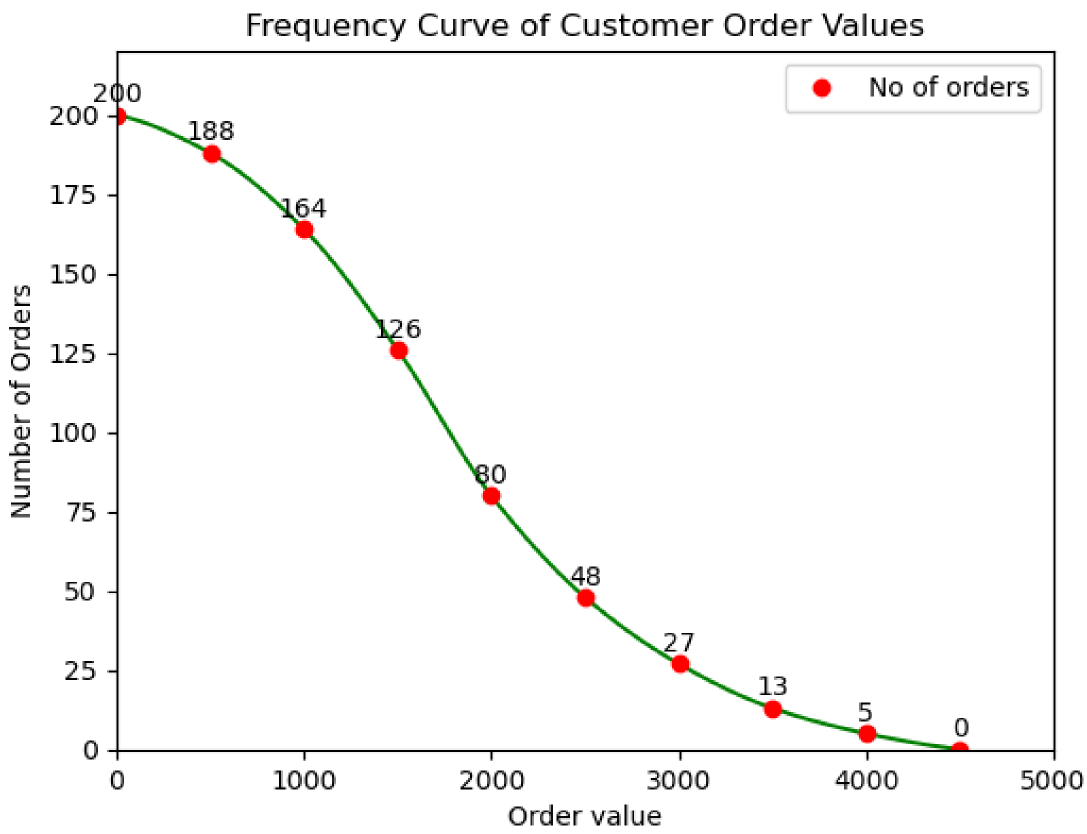
Question 5: More than Cumulative Frequency Curve:

Use same data above

[5]:

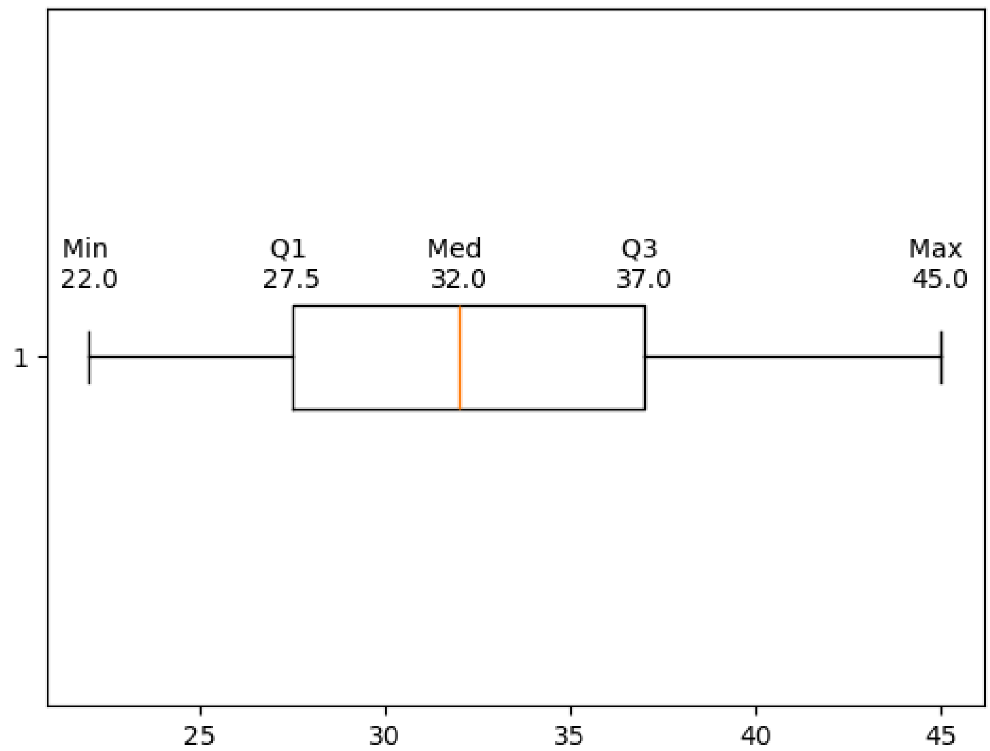
```
import matplotlib.pyplot as plt
import numpy as np
from scipy.interpolate import PchipInterpolator
lower = [ i for i in range (0,5000,500)]
orders = [12,24,38,46,32,21,14,8,5]
more_cf = np.cumsum(orders[::-1])[::-1]
more_cf = np.insert(more_cf,9,0)

#generate more x values for smooth curve
pchip = PchipInterpolator(lower,more_cf)
x_smooth = np.linspace(min(lower), max(lower), 300)
#create smooth curve
y_smooth = pchip(x_smooth)
plt.plot(x_smooth,y_smooth,color='green')
plt.plot(lower, more_cf, 'o',color='red',linestyle ='none',
         label ='No of orders')
for i in range(len(lower)):
    plt.text(lower[i], more_cf[i]+4, str(more_cf[i]), ha='center')
plt.xlabel('Order value')
plt.ylabel('Number of Orders')
plt.title('Frequency Curve of Customer Order Values')
plt.xlim(0,5000)
plt.ylim(0,220)
plt.legend()
plt.show()
```



#Question ^: box plot The folowing are the ages (in years) of 15 employees 22,24,25,27,28,30,31,32,34,34,36,38,40,42,45 Prepare a box plot and identify and the median and qu

[13]:
```
import matplotlib.pyplot as plt
import numpy as np
ages = [22,24,25,27,28,30,31,32,34,35,36,38,40,42,45]
Min ,Q1,Med, Q3 , Max = np.percentile(ages,[0,25,50,75,100])
Label = ["Min", "Q1" ,"Med","Q3","Max"]
value = [Min, Q1, Med, Q3,Max]
for i, Label in zip(value,Label):
    plt.text(i,1.1,f'{Label} \n{i}', ha ='center')
plt.boxplot(ages,vert =False)
plt.show()
```



[ ]:

[ ]: